

SDD — Software Design Document

BlockWallet API

Campo	Valor
Projeto	BlockWallet
Componente	blockwallet-api (Backend REST)
Versão	1.0.0
Data	2026-06-02
Autor	Aecio Pereira Santiago Junior
Framework	NestJS 11 + TypeScript 5
Runtime	Node.js 20+

Sumário

1. [Visão Geral](#)
2. [Arquitetura do Sistema](#)
3. [Stack Tecnológica](#)
4. [Estrutura de Pastas](#)
5. [Módulos da Aplicação](#)
6. [Referência Completa de Endpoints](#)
7. [Autenticação e Segurança](#)
8. [Integração com o Supabase](#)
9. [Integração com a CoinGecko API](#)
10. [Jobs Agendados \(Cron\)](#)
11. [Variáveis de Ambiente](#)
12. [Padrão de Resposta da API](#)
13. [Como Rodar Localmente](#)
14. [Como Fazer Deploy no Render](#)

1. Visão Geral

O **blockwallet-api** é o backend REST da aplicação BlockWallet. Ele atua como camada intermediária entre o app iOS e dois serviços externos: **Supabase** (banco de dados PostgreSQL gerenciado + autenticação) e **CoinGecko** (dados de mercado de criptomoedas em tempo real).

Responsabilidades principais

Responsabilidade	Descrição
Proxy inteligente da CoinGecko	Encaminhar chamadas ao CoinGecko e popular o cache <code>coin_metadata</code> no Supabase
Autenticação	Registrar, autenticar e validar usuários via Supabase Auth
Operações de carteira	Processar compras e vendas simuladas, delegando a lógica de saldo ao trigger SQL
Persistência de dados do usuário	Perfil, preferências, favoritos, snapshots de portfolio e alertas de preço
Jobs agendados	Sincronização diária de top 100 moedas e verificação periódica de alertas

O que a API não faz

- Não persiste preços em tempo real (somente snapshots pontuais)
- Não executa lógica de negócio de saldo — isso é delegado ao trigger `update_balance_on_transaction` do PostgreSQL
- Não serve assets de imagem diretamente — usa URLs do CDN da CoinGecko

2. Arquitetura do Sistema



Fluxo de uma requisição autenticada

App iOS

```
|
| 1. POST /api/v1/auth/sign-in { email, password }
| ← { accessToken, refreshToken }
|
| 2. GET /api/v1/wallet
|   Authorization: Bearer <accessToken>
|
| → SupabaseAuthGuard
|   | getUser(token) → Supabase Auth
|   | ✓ user injetado no request
|
| → WalletController → WalletService
|   | getClientWithToken(token) ← JWT propagado
|   | SELECT FROM v_wallet_with_metadata WHERE user_id = auth.uid()
|   | (RLS garante isolamento)
|
| ← { success: true, data: [...], timestamp }
```

3. Stack Tecnológica

Camada	Tecnologia	Versão
Runtime	Node.js	20+
Framework	NestJS	11
Linguagem	TypeScript	5
Banco de dados	Supabase (PostgreSQL 15)	—
Auth	Supabase Auth (JWT)	—
HTTP Client	Axios	1.x
Validação	class-validator + class-transformer	0.15.x
Agendamento	@nestjs/schedule (node-cron)	6.x
Rate Limiting	@nestjs/throttler	6.x
Gerenciador	npm	10+

4. Estrutura de Pastas

```

blockwallet-api/
├─ src/
│   └─ main.ts # Bootstrap: porta, CORS, ValidationPipe,
prefixo /api/v1
│   └─ app.module.ts # Módulo raiz – importa todos os módulos de
feature
│   └─ app.controller.ts # GET /health (público)
│   │
│   └─ config/
│       └─ configuration.ts # Mapeamento de variáveis de ambiente
│   │
│   └─ common/
│       └─ guards/
│           └─ supabase-auth.guard.ts # Guard JWT global via Supabase Auth
│       └─ decorators/
│           └─ current-user.decorator.ts # @CurrentUser(), @AccessToken()
│           └─ public.decorator.ts # @Public() – bypass do guard
│       └─ interceptors/
│           └─ transform.interceptor.ts # Envelope { success, data, timestamp }
│   │
│   └─ supabase/
│       └─ supabase.module.ts # @Global() – disponível em todos os
módulos
│       └─ supabase.service.ts # getAdminClient() |
getClientWithToken(jwt)
│   │
│   └─ coingecko/
│       └─ coingecko.module.ts
│       └─ coingecko.service.ts # Axios wrapper + cron de sync
│       └─ coingecko.types.ts # Interfaces: CoinMarket, CoinDetail,
SimplePrice, MarketChart
│   │
│   └─ auth/
│       └─ auth.module.ts
│       └─ auth.controller.ts # sign-up, sign-in, refresh, sign-out, me,
reset-password
│       └─ auth.service.ts
│       └─ dto/
│           └─ sign-up.dto.ts
│           └─ sign-in.dto.ts
│   │
│   └─ profile/

```

```

| | | └─ profile.module.ts
| | | └─ profile.controller.ts # GET/PATCH /profile, GET
/profile/dashboard
| | | └─ profile.service.ts
| | | └─ dto/update-profile.dto.ts
| |
| | └─ preferences/
| | | └─ preferences.module.ts
| | | └─ preferences.controller.ts # GET/PUT /preferences
| | | └─ preferences.service.ts
| | | └─ dto/update-preferences.dto.ts
| |
| |
| | └─ coins/
| | | └─ coins.module.ts
| | | └─ coins.controller.ts # Proxy CoinGecko + cache Supabase
| | | └─ coins.service.ts
| |
| |
| | └─ wallet/
| | | └─ wallet.module.ts
| | | └─ wallet.controller.ts # GET /wallet, POST /wallet/buy|sell
| | | └─ wallet.service.ts
| | | └─ dto/transaction.dto.ts
| |
| |
| | └─ transactions/
| | | └─ transactions.module.ts
| | | └─ transactions.controller.ts # GET /transactions, /transactions/summary
| | | └─ transactions.service.ts
| |
| |
| | └─ favorites/
| | | └─ favorites.module.ts
| | | └─ favorites.controller.ts # GET/POST/DELETE /favorites
| | | └─ favorites.service.ts
| | | └─ dto/add-favorite.dto.ts
| |
| |
| | └─ portfolio/
| | | └─ portfolio.module.ts
| | | └─ portfolio.controller.ts # GET/POST /portfolio/snapshots
| | | └─ portfolio.service.ts
| | | └─ dto/create-snapshot.dto.ts
| |
| |
| | └─ alerts/
| | | └─ alerts.module.ts
| | | └─ alerts.controller.ts # GET/POST/PATCH/DELETE /alerts

```

```

|   |   | alerts.service.ts           # Inclui cron a cada 5 min
|   |   | └─ dto/create-alert.dto.ts
|
|   |   |
|   |   |
|   |   | └─ dist/                     # Build compilado (gerado por `npm run
build`)
|   |   | └─ .env                     # Variáveis de ambiente (NÃO versionar)
|   |   | └─ .env.example             # Template das variáveis
|   |   | └─ nest-cli.json
|   |   | └─ package.json
|   |   | └─ tsconfig.json
|   |   | └─ SDD.md                   # Este documento

```

5. Módulos da Aplicação

5.1 SupabaseModule (Global)

Registrado com `@Global()`, disponível em toda a aplicação sem necessidade de importação explícita.

Método	Descrição
<code>getAdminClient()</code>	Retorna um <code>SupabaseClient</code> com <code>service_role</code> — ignora RLS . Usar apenas em crons, validações internas e triggers.
<code>getClientWithToken(jwt)</code>	Retorna um <code>SupabaseClient</code> que injeta o JWT do usuário no header. Respeita RLS — cada usuário acessa apenas seus próprios dados.

Por que dois clients? O `service_role` é necessário para jobs de background (sync de moedas, verificação de alertas) que precisam ler/escrever dados sem um usuário autenticado. O client com JWT garante que as políticas RLS do Supabase sejam respeitadas em todas as operações do usuário final.

5.2 CoingeckoModule

Encapsula todas as chamadas à CoinGecko API v3 via Axios. Configurado com timeout de 15 segundos. Se `COINGECKO_API_KEY` estiver definida, o header `x-cg-demo-api-key` é adicionado automaticamente.

5.3 AuthModule

Delega autenticação completamente ao Supabase Auth.

- **signUp** usa `auth.admin.createUser` com `email_confirm: true` (confirma imediatamente sem e-mail)
- **signIn** usa `auth.signInWithPassword`, retorna `accessToken` (JWT) + `refreshToken`

- `refreshToken` renova a sessão via `auth.refreshSession`
- `signOut` revoga o token ativo no Supabase

5.4 WalletModule

Operações de compra/venda são inseridas diretamente na tabela `transactions`. A lógica de saldo (débito/crédito) e a atualização de `wallet_items` (posição consolidada com média ponderada) são executadas pelo trigger PostgreSQL `update_balance_on_transaction`, garantindo atomicidade sem lógica duplicada no backend.

Fluxo de compra:

```
POST /wallet/buy
|
├─ ensureCoinExists(cryptoId) → admin client (verifica coin_metadata)
|
├─ INSERT INTO transactions → user client (RLS)
   type: 'buy', quantity, price_at_transaction
   |
   └─ TRIGGER update_balance_on_transaction (PostgreSQL)
       ├── Verifica saldo disponível
       ├── Debita simulated_balance_usd em profiles
       ├── Upsert em wallet_items (média ponderada)
       └─ Preenche balance_before_usd / balance_after_usd
```

5.5 AlertsModule

Além dos endpoints CRUD, o `AlertsService` possui um cron que:

1. Busca todos os alertas com `is_active = true` via admin client
2. Agrupa os `crypto_id` únicos e chama `GET /simple/price` na CoinGecko
3. Compara cada preço atual com `target_price + alert_type`
4. Atualiza os alertas disparados: `is_active = false`, `triggered_at = now()`

6. Referência Completa de Endpoints

Prefixo global: `/api/v1`

Autenticação: header `Authorization: Bearer <accessToken>` em todos os endpoints, exceto os marcados com  (público).

6.1 Health

Método	Rota	Auth	Descrição
GET	/health		Retorna status da API

Resposta:

```
{
  "success": true,
  "data": {
    "status": "ok",
    "service": "blockwallet-api",
    "timestamp": "2026-06-02T10:00:00.000Z"
  },
  "timestamp": "2026-06-02T10:00:00.000Z"
}
```

6.2 Auth — /auth

Método	Rota	Auth	Body	Descrição
POST	/auth/sign-up		{ email, password, displayName? }	Cadastro
POST	/auth/sign-in		{ email, password }	Login
POST	/auth/refresh		{ refreshToken }	Renovar tokens
POST	/auth/sign-out		—	Encerrar sessão
POST	/auth/reset-password		{ email }	Solicitar recuperação de senha
POST	/auth/me		—	Dados do usuário autenticado

Resposta de sign-in:

```
{
  "success": true,
  "data": {
    "accessToken": "eyJ...",
    "refreshToken": "eyJ...",
    "expiresAt": 1748905200,
    "user": { "id": "uuid", "email": "user@email.com" }
  },
  "timestamp": "..."
}
```

6.3 Profile — /profile

Método	Rota	Auth	Body	Descrição
GET	/profile		—	Perfil completo do usuário
PATCH	/profile		{ displayName?, avatarUrl? }	Atualizar perfil
GET	/profile/dashboard		—	Resumo do dashboard (1 query via RPC)

Resposta de /profile/dashboard:

```
{
  "success": true,
  "data": {
    "simulated_balance_usd": "9500.00",
    "total_invested_usd": "1250.00",
    "asset_count": "3",
    "transaction_count": "5",
    "favorite_count": "7",
    "crypto_ids": ["bitcoin", "ethereum", "solana"]
  },
  "timestamp": "..."
}
```

6.4 Preferences — /preferences

Método	Rota	Auth	Body	Descrição
GET	/preferences		—	Preferências de UI do usuário
PUT	/preferences		UpdatePreferencesDto	Upsert das preferências

Body de PUT (todos os campos opcionais):

```
{
  "marketSortOrder": "market_cap_desc",
  "chartPeriodDefault": "7",
  "currencyDisplay": "usd",
  "theme": "dark",
  "showPortfolioValue": true,
  "priceChangeTimeframe": "24h",
  "notificationsEnabled": true
}
```

6.5 Coins — /coins

Método	Rota	Auth	Query Params	Descrição
GET	/coins/markets		page , per_page , order , vs_currency	Lista de mercado (CoinGecko)
GET	/coins/price/simple		ids , vs_currencies	Preço atual (tempo real)
GET	/coins/search/query		q	Busca por nome/símbolo
GET	/coins/:id		—	Detalhe completo da moeda
GET	/coins/:id/chart		days , vs_currency	Dados históricos do gráfico
GET	/coins/cache/list		page , per_page	Cache local (Supabase, sem hit na CoinGecko)
GET	/coins/cache/:id		—	Metadado em cache de uma moeda

Valores válidos para order : market_cap_desc , market_cap_asc , volume_desc , volume_asc , price_change_desc , id_asc

Valores válidos para days : 1 , 7 , 30 , 90 , 365 , max

6.6 Wallet — /wallet

Método	Rota	Auth	Body	Descrição
GET	/wallet		—	Todos os ativos da carteira
GET	/wallet/:cryptoId		—	Posição de uma moeda específica
POST	/wallet/buy		BuyDto	Comprar uma criptomoeda
POST	/wallet/sell		SellDto	Vender uma criptomoeda

Body de buy/sell:

```
{
  "cryptoId": "bitcoin",
  "quantity": 0.01,
  "priceAtTransaction": 98500.00,
  "notes": "Compra inicial"
}
```

priceAtTransaction deve ser obtido pelo app via GET /coins/price/simple?ids=bitcoin **imediatamente antes** de chamar este endpoint, para garantir o preço correto no momento da operação.

Erros possíveis:

- 400 Bad Request — Saldo insuficiente (buy) / Quantidade insuficiente (sell)
- 400 Bad Request — Moeda não encontrada no cache (chamar GET /coins/:id primeiro)

6.7 Transactions — /transactions

Método	Rota	Auth	Query Params	Descrição
GET	/transactions		crypto_id?, type?, page, per_page	Histórico paginado
GET	/transactions/summary		—	Total comprado/vendido em USD
GET	/transactions/:id		—	Transação individual

Valores válidos para type: buy, sell

Resposta de `/transactions`:

```
{
  "success": true,
  "data": {
    "data": [
      {
        "id": "uuid",
        "crypto_id": "bitcoin",
        "crypto_name": "Bitcoin",
        "crypto_symbol": "btc",
        "type": "buy",
        "quantity": "0.01000000000",
        "price_at_transaction": "98500.00000000",
        "total_value_usd": "985.00",
        "balance_before_usd": "10000.00",
        "balance_after_usd": "9015.00",
        "created_at": "2026-06-02T..."
      }
    ],
    "total": 5,
    "page": 1,
    "perPage": 20
  },
  "timestamp": "..."
}
```

6.8 Favorites — `/favorites`

Método	Rota	Auth	Body	Descrição
GET	<code>/favorites</code>		—	Lista de favoritos com metadados
POST	<code>/favorites</code>		<code>{ cryptoId</code> <code>}</code>	Adicionar favorito
DELETE	<code>/favorites/:cryptoId</code>		—	Remover favorito
GET	<code>/favorites/:cryptoId/check</code>		—	Verificar se é favorito

6.9 Portfolio — /portfolio

Método	Rota	Auth	Body / Params	Descrição
GET	/portfolio/snapshots		days (default: 30)	Snapshots do período
GET	/portfolio/snapshots/latest		—	Snapshot mais recente
POST	/portfolio/snapshots		CreateSnapshotDto	Gravar snapshot do dia

Body de POST:

```
{
  "totalValueUsd": 1350.00,
  "availableBalanceUsd": 8650.00,
  "assetCount": 3,
  "snapshotDate": "2026-06-02"
}
```

O app deve chamar POST /portfolio/snapshots **uma vez por dia**, na primeira abertura, com os valores calculados a partir dos preços atuais da CoinGecko.

6.10 Alerts — /alerts

Método	Rota	Auth	Body	Descrição
GET	/alerts		—	Todos os alerts (?active=true para filtrar)
POST	/alerts		CreateAlertDto	Criar alerta
PATCH	/alerts/:id/deactivate		—	Desativar alerta
DELETE	/alerts/:id		—	Remover alerta

Body de POST:

```
{
  "cryptoId": "bitcoin",
  "alertType": "above",
  "targetPrice": 100000.00
}
```

Valores válidos para `alertType`: `above` (notificar quando preço > target), `below` (notificar quando preço < target)

7. Autenticação e Segurança

7.1 SupabaseAuthGuard

Aplicado **globalmente** via `APP_GUARD`. Para cada requisição:

1. Verifica se a rota é pública (`@Public()` decorator) — se sim, bypassa
2. Extrai o `Bearer <token>` do header `Authorization`
3. Chama `supabase.auth.getUser(token)` para validar o JWT
4. Injeta `user` e `accessToken` no objeto `request` para uso nos controllers

```
// Rotas públicas – bypass do guard
@Public()
@Get('markets')
getMarkets() { ... }

// Rotas protegidas – guard valida JWT automaticamente
@Get()
getWallet(@CurrentUser() user: User, @AccessToken() token: string) { ... }
```

7.2 Row Level Security (RLS)

O Supabase possui políticas RLS em todas as tabelas de usuário. O `getClientWithToken(jwt)` propaga o JWT para o Supabase, que resolve `auth.uid()` e aplica as políticas automaticamente. Isso significa que:

- Um usuário **nunca** pode acessar dados de outro usuário, mesmo que passe um `user_id` diferente na query
- Toda a segurança de isolamento de dados é garantida pelo banco — não pelo backend

7.3 Rate Limiting

`ThrottlerGuard` aplicado globalmente: **60 requisições por minuto por IP**.

Configurável via env vars:

```
THROTTLE_TTL=60000    # janela em ms
THROTTLE_LIMIT=60     # máximo de requests na janela
```

7.4 Validação de Entrada

`ValidationPipe` global com:

- `whitelist: true` — remove campos não declarados nos DTOs
- `forbidNonWhitelisted: true` — retorna `400` se houver campos extras
- `transform: true` — converte strings de query params para tipos corretos (ex: `'7'` → `7`)

8. Integração com o Supabase

8.1 Duas estratégias de cliente

```
SupabaseService
├─ getClient()           service_role key → ignora RLS
│   Uso: crons, validações internas, upsert de coin_metadata
│
├─ getClientWithToken(jwt) anon key + JWT header → respeita RLS
│   Uso: todos os endpoints de usuário
```

8.2 RPC Functions chamadas

Function	Chamada em	Descrição
<code>upsert_coin_metadata(...)</code>	CoingeckoService (cron + detalhe)	Insere/atualiza metadados da moeda
<code>get_dashboard_summary(user_id)</code>	ProfileService	Resumo do dashboard em 1 query

8.3 Views utilizadas

View	Usada em	Descrição
<code>v_wallet_with_metadata</code>	WalletService	Carteira + nome/símbolo/imagem da moeda
<code>v_transactions_with_metadata</code>	TransactionsService	Histórico + metadados da moeda
<code>v_favorites_with_metadata</code>	FavoritesService	Favoritos + rank/imagem da moeda

8.4 Chaves necessárias

Chave	Onde obter	Uso
<code>SUPABASE_URL</code>	Dashboard > Settings > API > Project URL	Base URL do projeto
<code>SUPABASE_ANON_KEY</code>	Dashboard > Settings > API > anon public	Client de usuário (JWT)

SUPABASE_SERVICE_ROLE_KEY	Dashboard > Settings > API > service_role	Client admin (crons)
---------------------------	---	----------------------

9. Integração com a CoinGecko API

9.1 Endpoints consumidos

Endpoint CoinGecko	Método da API	Descrição
GET /coins/markets	getMarkets()	Lista de moedas com dados de mercado
GET /coins/{id}	getCoinDetail()	Detalhe completo de uma moeda
GET /simple/price	getSimplePrice()	Preço atual de uma ou mais moedas
GET /coins/{id}/market_chart	getMarketChart()	Dados históricos para gráfico
GET /search	searchCoins()	Busca por nome/símbolo

9.2 Estratégia de cache

A API usa o Supabase como **cache persistente** para `coin_metadata`:

```
Requisição GET /coins/:id
|
├─ CoinGecko API (resposta ao cliente)
|
└─ syncCoinDetail(id) ← executa em background (fire-and-forget)
    └─ upsert_coin_metadata() no Supabase
```

O cache não é verificado antes de chamar a CoinGecko — dados do endpoint `/coins` sempre vêm ao vivo. O cache serve como dado offline para:

- Foreign key de `wallet_items.crypto_id` → `coin_metadata.coin_id`
- Views `v_wallet_with_metadata`, `v_favorites_with_metadata`
- Listagem offline via `/coins/cache/list`

9.3 Rate limits da CoinGecko

Tier	Limite	Header da chave
Demo (gratuito)	30 req/min	x-cg-demo-api-key
Pro	Variável por plano	x-cg-pro-api-key

Se nenhuma chave for configurada, as chamadas são feitas sem autenticação (limite público da CoinGecko, ~10-15 req/min).

10. Jobs Agendados (Cron)

10.1 Sync diário de moedas

```
CoinGeckoService.syncTopCoins()  
Agendamento: EVERY_DAY_AT_6AM (0 6 * * *)
```

1. Calcula quantas páginas precisam ser buscadas (`syncTopN / 250`)
2. Busca lotes de até 250 moedas por página no endpoint `/coins/markets`
3. Para cada moeda, chama `upsert_coin_metadata()` via admin client
4. Log: "Sync concluído: N moedas atualizadas"

Configurável: `COINGECKO_SYNC_TOP_N=100` (default)

10.2 Verificação de alertas de preço

```
AlertsService.checkAlerts()  
Agendamento: */5 * * * * (a cada 5 minutos)
```

1. Busca todos os `price_alerts` com `is_active = true` via admin client
2. Deduplica os `crypto_id` e chama `getSimplePrice()`
3. Compara `current_price` × `alert_type` × `target_price`
4. Atualiza alertas disparados: `is_active = false`, `triggered_at = now()`

Notificações push: a versão atual apenas marca o alerta como disparado. A entrega de push notifications ao dispositivo iOS deve ser implementada como extensão futura (ex: Supabase Edge Functions + APNs).

11. Variáveis de Ambiente

Crie um arquivo `.env` na raiz de `blockwallet-api/` com base no `.env.example`:

Variável	Obrigatória	Default	Descrição
<code>PORT</code>	Não	<code>3000</code>	Porta HTTP do servidor

CORS_ORIGINS	Não	*	Origens permitidas (CSV)
SUPABASE_URL	Sim	—	URL do projeto Supabase
SUPABASE_ANON_KEY	Sim	—	Chave pública anon
SUPABASE_SERVICE_ROLE_KEY	Sim	—	Chave secreta service_role
SUPABASE_ACCESS_TOKEN	Não	—	PAT para Management API
COINGECKO_BASE_URL	Não	https://api.coingecko.com/api/v3	Base URL da CoinGecko
COINGECKO_API_KEY	Não	—	Chave Demo/Pro da CoinGecko
COINGECKO_SYNC_TOP_N	Não	100	Top N moedas no cron diário
THROTTLE_TTL	Não	60000	Janela de rate limit (ms)
THROTTLE_LIMIT	Não	60	Máx. requisições por janela

Segurança: SUPABASE_SERVICE_ROLE_KEY tem acesso irrestrito ao banco. Nunca expô-la no frontend, em logs ou em repositórios públicos.

12. Padrão de Resposta da API

Todas as respostas são envolvidas pelo TransformInterceptor :

```
// Sucesso
{
  "success": true,
  "data": <payload>,
  "timestamp": "2026-06-02T10:00:00.000Z"
}

// Erro (gerado pelo NestJS ExceptionFilter padrão)
{
  "statusCode": 400,
  "message": "Saldo insuficiente para realizar a compra.",
  "error": "Bad Request"
}
```

Códigos de status utilizados

Código	Situação
200	Sucesso (GET, PUT, PATCH, POST sem criação)
201	Recurso criado (POST com insert)
400	Validação de entrada ou regra de negócio
401	Token ausente, inválido ou expirado
404	Recurso não encontrado
409	Conflito (ex: favorito duplicado)
429	Rate limit excedido
500	Erro interno

13. Como Rodar Localmente

Pré-requisitos

- Node.js **20+** — verificar: `node --version`
- npm **10+** — verificar: `npm --version`
- Projeto Supabase criado com o schema `docs/blockwallet_schema.sql` aplicado
- (Opcional) Chave da CoinGecko para aumentar o rate limit

Passo 1 — Aplicar o schema no Supabase

Acesse o [Supabase Dashboard](#), abra o projeto, vá em **SQL Editor** e execute o conteúdo de `docs/blockwallet_schema.sql`.

Se o schema já foi aplicado, pule este passo.

Passo 2 — Obter as chaves do Supabase

No dashboard do projeto:

```
Settings → API
├─ Project URL          → SUPABASE_URL
├─ anon public          → SUPABASE_ANON_KEY
└─ service_role secret → SUPABASE_SERVICE_ROLE_KEY
```

Passo 3 — Configurar as variáveis de ambiente

```
# Dentro de blockwallet-api/
cp .env.example .env
```

Edite `.env` e preencha as três variáveis obrigatórias:

```
SUPABASE_URL=https://abcdefghijkl.supabase.co
SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Passo 4 — Instalar dependências

```
cd blockwallet-api
npm install
```

Passo 5 — Rodar em modo desenvolvimento

```
npm run start:dev
```

A API estará disponível em: `http://localhost:3000/api/v1`

Confirme que está rodando:

```
curl http://localhost:3000/api/v1/health
# {"success":true,"data":{"status":"ok","service":"blockwallet-api",...}}
```

Passo 6 — (Opcional) Rodar o sync inicial de moedas manualmente

O cron diário executa às 06:00 UTC. Para popular o cache imediatamente sem esperar, chame qualquer endpoint de detalhe de moeda:

```
# Após se autenticar e obter um token:
curl http://localhost:3000/api/v1/coins/bitcoin
curl http://localhost:3000/api/v1/coins/ethereum
```

Ou aguardar o cron (dispara às 06:00 UTC) que populará as top 100 automaticamente.

Scripts disponíveis

```
npm run start:dev    # Hot-reload (desenvolvimento)
npm run start:prod   # Inicia a build compilada (node dist/main)
npm run build        # Compila TypeScript → dist/
npm run lint         # ESLint com auto-fix
```

Testando os endpoints

1. Cadastrar um usuário:

```
curl -X POST http://localhost:3000/api/v1/auth/sign-up \
-H "Content-Type: application/json" \
-d '{"email":"teste@email.com","password":"senha123","displayName":"Teste"}'
```

2. Fazer login:

```
curl -X POST http://localhost:3000/api/v1/auth/sign-in \
-H "Content-Type: application/json" \
-d '{"email":"teste@email.com","password":"senha123"}'
# Guarde o accessToken retornado
```

3. Ver o dashboard:

```
curl http://localhost:3000/api/v1/profile/dashboard \
-H "Authorization: Bearer <accessToken>"
```

4. Buscar preço atual e comprar:

```
# Passo a) Buscar preço atual
curl "http://localhost:3000/api/v1/coins/price/simple?ids=bitcoin"

# Passo b) Buscar detalhes (popula o cache coin_metadata)
curl http://localhost:3000/api/v1/coins/bitcoin

# Passo c) Comprar
curl -X POST http://localhost:3000/api/v1/wallet/buy \
  -H "Authorization: Bearer <accessToken>" \
  -H "Content-Type: application/json" \
  -d '{"cryptoId":"bitcoin","quantity":0.001,"priceAtTransaction":98500}'
```

14. Como Fazer Deploy no Render

O [Render](#) é uma plataforma de hosting que suporta serviços Node.js com deploys automáticos via Git. O plano gratuito (Hobby) é suficiente para desenvolvimento e testes.

Pré-requisitos

- Conta no [Render](#)
- Código em repositório Git (GitHub, GitLab ou Bitbucket)
- Schema aplicado no Supabase

Passo 1 — Preparar o repositório

Certifique-se que o `.gitignore` exclui arquivos sensíveis:

```
# Verificar se .env está ignorado
cat blockwallet-api/.gitignore | grep .env
# Deve aparecer: .env
```

Faça o commit e push do projeto `blockwallet-api/` para o repositório.

Passo 2 — Criar o Web Service no Render

1. Acesse dashboard.render.com e clique em **New + → Web Service**
2. Conecte seu repositório Git e selecione o repositório do BlockWallet
3. Configure o serviço:

Campo	Valor
Name	blockwallet-api

Region	Ohio (US East) ou Frankfurt (EU)
Branch	main
Root Directory	blockwallet-api
Runtime	Node
Build Command	npm install && npm run build
Start Command	npm run start:prod
Instance Type	Free (ou Starter para produção)

Root Directory é essencial se o repositório contém múltiplos projetos (como o iOS app e a API). Aponte para a pasta `blockwallet-api`.

Passo 3 — Configurar as variáveis de ambiente no Render

Na tela do serviço, clique em **Environment** e adicione as variáveis:

Key	Value
NODE_ENV	production
SUPABASE_URL	https://<project-ref>.supabase.co
SUPABASE_ANON_KEY	eyJ...
SUPABASE_SERVICE_ROLE_KEY	eyJ...
COINGECKO_API_KEY	CG-... (opcional)
COINGECKO_SYNC_TOP_N	100
CORS_ORIGINS	URL do app ou * para desenvolvimento
THROTTLE_TTL	60000
THROTTLE_LIMIT	60

PORT **não precisa** ser configurado — o Render define a porta automaticamente via a variável interna PORT.

SUPABASE_SERVICE_ROLE_KEY deve ser marcada como **Secret** no Render (ícone de cadeado) para não aparecer nos logs.

Passo 4 — Deploy

Clique em **Create Web Service**. O Render irá:

- 1. Clonar o repositório
- 2. Executar `npm install && npm run build` (compila o TypeScript)
- 3. Iniciar o processo com `npm run start:prod` (executa `node dist/main`)

Acompanhe os logs em tempo real na aba **Logs** do serviço.

URL do serviço após deploy:

```
https://blockwallet-api.onrender.com/api/v1
```

Confirme que está funcionando:

```
curl https://blockwallet-api.onrender.com/api/v1/health
```

Passo 5 — Deploy automático (CI/CD)

Por padrão, o Render faz **redesploy automático** a cada push para a branch configurada (`main`). Para desabilitar, vá em **Settings** → **Auto-Deploy** → `No`.

Passo 6 — Domínio personalizado (opcional)

Em **Settings** → **Custom Domains**, adicione seu domínio. O Render provisiona o certificado TLS (HTTPS) automaticamente via Let's Encrypt.

Considerações para produção

Item	Recomendação
Plano Render	<code>Starter</code> (\$7/mês) para evitar cold starts do plano gratuito
Cold start	No plano gratuito, o serviço dorme após 15 min de inatividade. Use um serviço de ping (ex: UptimeRobot) para mantê-lo ativo
Logs	Render mantém 7 dias de logs. Para persistência maior, configure um log drain (ex: Papertrail)
Health check	Configure em Settings → Health Check Path → <code>/api/v1/health</code>
Scaling	Para múltiplas instâncias, os crons do <code>@nestjs/schedule</code> executarão em cada instância . Migre os crons para Supabase Edge Functions ou Render Cron Jobs em produção com escala horizontal

Render Cron Jobs (alternativa para crons em escala)

Para evitar execução duplicada dos jobs em múltiplas instâncias, crie serviços separados do tipo **Cron Job** no Render:

Sync diário de moedas:

```
Name: blockwallet-coin-sync
Schedule: 0 6 * * *
Command: node -e "require('./dist/coingecko/coingecko.service').syncTopCoins()"
```

Esta abordagem requer refatoração do serviço para execução standalone. Para projetos com instância única, os crons internos do NestJS são suficientes.

Apêndice — Diagrama de tabelas relacionadas à API

